# ECE 315 Lab 2: Interfacing to Input/Output Devices Using UART

Authors: Yaaqoob Choulli, Rowan Rasmusson

Student ID: 1752634, 1757167

CCID: yaaqoob, rrasmuss

Course: ECE 315

Lab Section: H21

Date Performed: March 3, 2026

Date Due: March 10, 2026

**Abstract:**
The goal of the second lab in computer interfacing is to familiarize ourselves with Universal Asynchronous Receiver-Transmitter (UART) communication and apply FreeRTOS queues for inter-task communication. The objectives to complete this were to implement UART interfacing for user input and system output between the PC host and the Zybo Z7 board. These objectives were first implemented using a polling loop method to generate a SHA-256 hash string. Later, we extended our system from Lab 1 to allow UART command inputs to control the seven-segment display and the LED. Finally, we implemented a robust interrupt-driven UART driver to improve system efficiency and responsiveness. The hardware this was implemented on was a Zybo Z7-10 development board on a Zynq 7000 processor.

**Design:**

**Part 1:**

The objective for part 1 was to implement a hashing application that takes user-provided username and password information and generates a hash string using the SHA-256 algorithm. This was achieved through a polling loop interface to receive input from the UART device and send data out using FreeRTOS queues. We utilized q_rx_byte to receive incoming characters and q_tx to transmit characters back out to the terminal.

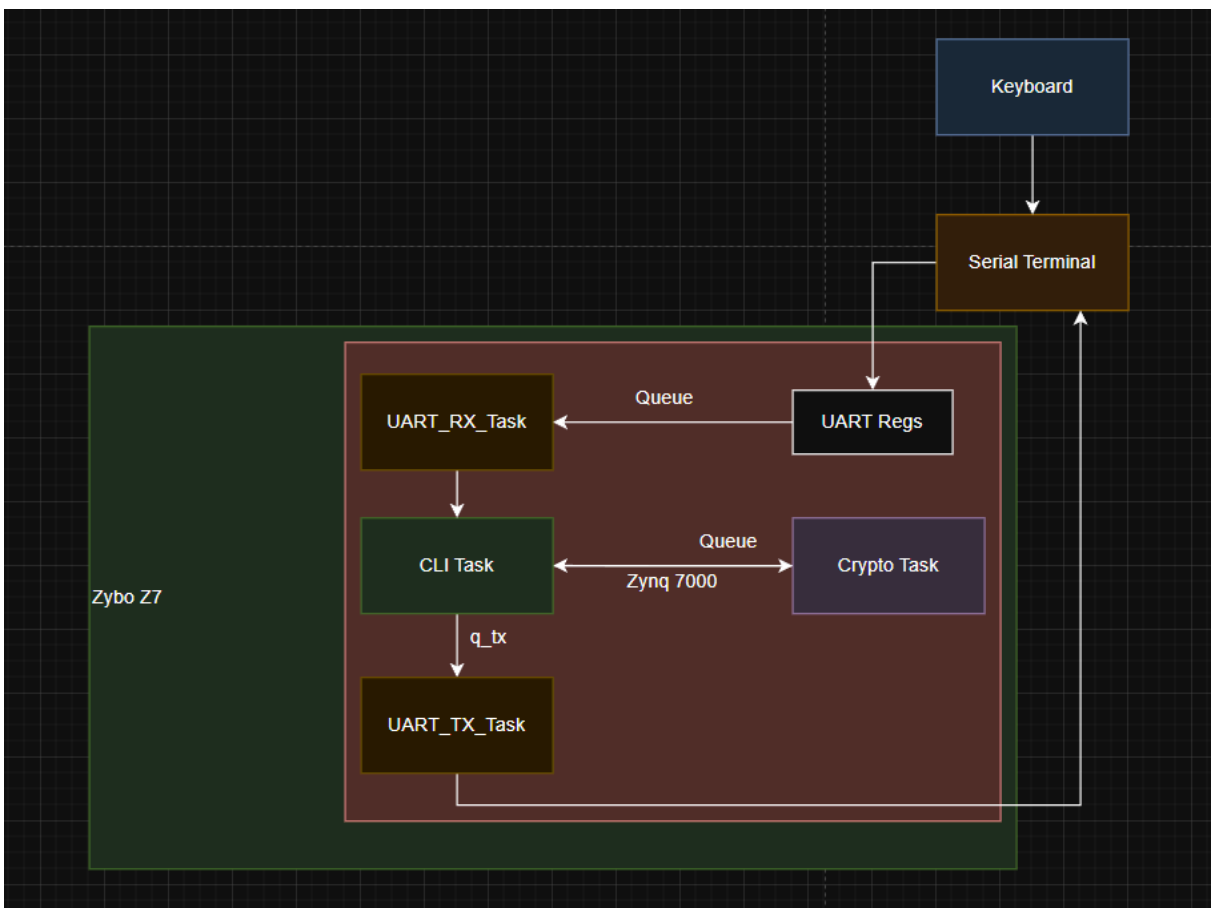


**Figure 1**: This picture shows an architectural diagram of the components in action for part 1. Shows is a full circle implementation that uses a UART layer to receive and print data to/from the serial terminal. The input goes to the registers, to the UART_RX_Task, then it is handled and processed using the CLI

and Crypto tasks. This is where the data is manipulated, and calculations are performed. Finally, the result is passed to the UART_TX_Task. This sends it back to the serial terminal to be displayed to the user, specifically through our UART layer.

**Part 2:**

Part 2 was a continuation of our work in Lab 1. Basically, we extended the functionality of part 1 to include adjusting the LED and writing to the SSD. This is why things like Crypto Task and CLI tasks are still included in this part. We integrated our previous peripheral control tasks (adjusting the RGB LED and writing to the SSD) with the new UART communication system. By building onto the existing architecture, we enabled the application to take text commands via UART to dictate the LED brightness or SSD values, decoupling the data producer tasks from the consumer tasks using dedicated FreeRTOS queues. Another design decision we needed to make was how to perform the same functionality from lab 1 part 3 when the input was taken from the UART path. To achieve this, we decided to save time by copying over code from our old lab and pasting it in the appropriate handler portion in the switch case where the specific functionality was handled. This design decision enabled us to complete the lab on time.

**Part 3:**

In part 3, the design shifted from a polling-based interface to an interrupt-driven UART driver. Unlike polling, which requires continuous CPU involvement, hardware interrupts allow the system to react asynchronously to incoming data. The driver's core responsibility is managing two distinct queues: one for incoming data, *xRxQueue* and one for outgoing data, *xTxQueue*.

When a byte is received, the *UART* generates an interrupt signal, which temporarily halts the executing software to run the Interrupt Service Routine (ISR). The ISR captures the incoming bytes and pushes them to *xRxQueue*, where they are later retrieved by *myReceiveByte()*.

For transmission, *mySendByte()* handles the logic. If *xTxQueue* is empty, it sends a single byte directly to the *UART* FIFO (lines 142-145). This initial byte is crucial because it triggers the *TXEMPTY* interrupt once it finishes transmitting. Once triggered, the ISR takes over (lines 146-149), pulling the remaining bytes from *xTxQueue* and loading them onto the hardware FIFO to be sent. These actions can be seen in the code below in Figure 2.

```
        taskENTER_CRITICAL();
        // Checking the FIFO MAY cause an unwanted TX interrupt
        if (uxQueueMessagesWaiting(xTxQueue) == 0) {
            XUartPs_WriteReg(UART_BASEADDR, UART_FIFO_OFFSET, data);
            // Enaable interrupt to transmit via ISR
            enableTxEmpty();
        } else {
            xQueueSend(xTxQueue, &data, 0);
            enableTxEmpty();
        }
        taskEXIT_CRITICAL();
        return;
    }
```

**Figure 2:** This figure shows our send_byte function from part 3. It checks the *xTxQueue* if there is data waiting to be sent in the queue, if not it writes to the UART FIFO and enables an interrupt to signal that there is data to send. If the queue already has data in it, it simply puts the data it wants to send into the back of the queue and still enables an interrupt. Either way, it makes sure that data is transmitted by enabling the TxEmpty interrupt.

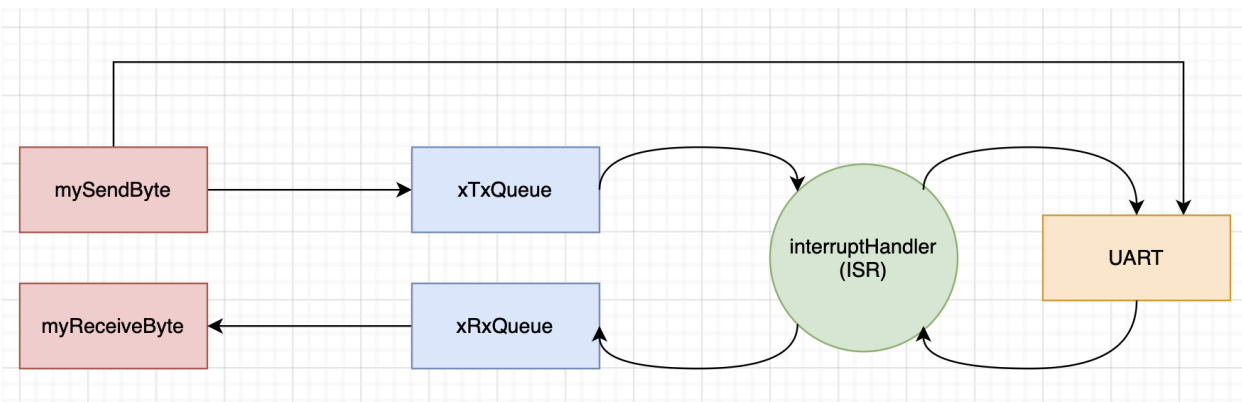


**Figure 3**: This figure illustrates the communication flow between the application tasks and the UART. Data is exchanged through *xRxQueue* and *xTxQueue*, allowing the system to handle data asynchronously. The Interrupt Service Routine (ISR) acts as the bridge, moving bytes between these software queues and the internal *UART* FIFOs. We can also see the direct line from the *mySendByte* to the *UART*, this is for the first initial byte as discussed above.

## <u>Discussion:</u>

**Part 1:**

During the implementation of our polling loop, we ran into a few minor difficulties. Initially, we noticed that our q_tx queue was not transmitting correctly; we discovered it was never initialized, which we quickly fixed. Furthermore, our UART receive loop (UART_RX) was initially missing a delay, causing it to default to busy-waiting. This is poor practice in a RTOS environment because it hogs the CPU's time slice. Lastly, we resolved an issue with the UART transmit by updating the xTicksToWait parameter from

0xFFFF to 0. A blocking call is not appropriate in a polling loop because it holds the CPU indefinitely, stalling other equal-priority tasks.

**Part 2:**
The integration of our Lab 1 components went smoothly. By leveraging FreeRTOS queues (q_cmd), we were able to parse commands arriving from the UART interface and route them to their respective hardware driver tasks. Routing hardware calls through these driver tasks is important because it maintains architectural separation, keeping peripheral logic decoupled from string parsing logic. An issue worth mentioning is that our part 2 repeats a lot of code from our lab1 part3. This is clearly not ideal as this is not best coding practice and results in large chunks of repeated code that shouldn't need to be in multiple places in the same file. To make the code cleaner, we would ideally have used additional FreeRTOS queues to send the input data to our vButtonsTask and the vSKeypadTask so that we would not have to repeat the logic that is already implemented there. This would have resulted in cleaner and easier to follow code. But at the end of the day, we wanted to make something that worked within the time period we had in the lab.

**Part 3:**
During the implementation of Part 3, we encountered one main issue where our TX interrupt count unexpectedly became higher than our RX interrupt count when inputting characters slowly. Initially, our mySendByte() function checked if the hardware UART FIFO was empty (XUartPs_IsTransmitFifoEmpty) before writing a byte directly to it and enabling the interrupt impacting our Tx count.

After discussing the issue with our lab instructor, we realized that checking the FIFO directly was causing unwanted TX interrupts. The lab instructor recommended checking if our software transmission queue (xTxQueue) was empty instead. By updating our critical section to check if uxQueueMessagesWaiting(xTxQueue) is equal to 0, we ensured that the initial byte was only written to the FIFO if the queue was entirely empty. If the queue had data, the byte was simply appended to the queue, properly relying on the ISR to drain it. We also ensured the enableTxEmpty() function was called regardless of the branch taken. This adjustment perfectly aligned our TX interrupt counts with expected behaviours.

It is important to note that we did not observe this issue when sending massive amounts of input, such as an entire source code file. We determined the reason for this was the sheer volume and speed of the data transfer. With a large file, the xTxQueue fills up almost instantly. Because the queue was constantly non-empty, the logic consistently defaulted to the else branch (sending to the queue), which is the correct behaviour for high-throughput data. The bug was only "exposed" during slow manual typing, where the FIFO had enough time to empty between keystrokes while the software was still processing, leading to the inconsistent interrupt counts.

```
238    void mySendByte(u8 data)
239    {
240        taskENTER_CRITICAL();
241        if (XUartPs_IsTransmitFifoEmpty(UART_BASEADDR)) {
242            XUartPs_WriteReg(UART_BASEADDR, UART_FIFO_OFFSET, data);
243            // Enaable interrupt to transmit via ISR
244            enableTxEmpty();
245        } else {
246            xQueueSend(xTxQueue, &data, 0);
247        }
248        taskEXIT_CRITICAL();
249        return;
250    }
```

**Figure 4**: Initial problem code, the issue discussed above can be seen on line 241.

**<u>Testing Methodology:</u>**

For testing, we manually tried different combinations of inputs on the keypad and buttons controlling the LED. All these tests passed. Some test combinations for the SSD and LED include:

**Table 1: Test cases for Part 3. <u>Disclaimer</u>: Note, '\r # \r' is pressed after all tests**

| Test | Expected Output | Rationale |
|---|---|---|
| Send a single upper and lowercase letter (e.g. 'a' and 'Z') | Echo of 'A' and 'z' (upper and lower cased) appears in terminal. RX = 5, TX = 5. | Verifies that one RX interrupt fires per received byte and one TX interrupt fires per echoed byte in addition to case-swap logic. |
| Send a single digit (e.g. '5') | Echo of '5' (unchanged) appears in terminal. RX = 4, TX = 4. | Verifies that non-alphabetic characters pass through without modification and still generate correct interrupt counts. |
| Send a single special character (e.g. '@') | Echo of '@' appears in terminal. RX = 4, TX = 4. | Ensures the driver does not mishandle special ASCII characters outside the alpha range. |
| Send only '\r # \r' | No echo output<br>RX = 3, TX = 3. | Validates that the values are flushed and not held in the buffers/queues |

| | | |
|---|---|---|
| Send a mixed-case 5-character word (e.g. 'HeLLo') | Echo 'hEllO' in terminal. RX = 8, TX = 8. | Confirms that the case-swap logic handles alternating uppercase and lowercase correctly within a single word. |
| Send 5 characters then display stats with \r#\r (byte-by-byte mode) | RX interrupts = 8 (5 chars + 3 for sequence). TX interrupts = 8. | Confirms RX and TX counts remain equal in byte-by-byte mode, validating that each received byte produces exactly one echoed byte. |
| Send a 20-character string via string mode (sendMethod = 1) then display stats | RX = 23 (20 + \r + \r#\r). TX interrupts = 1 for the string burst + 1 for stats display. | Demonstrates that mySendString() causes fewer TXEMPTY interrupts than individual mySendByte() calls, since the ISR drains the entire TX queue in one interrupt. |
| Send large file with many characters. The file we chose was lab2_part3.c | Bytes received: 11089<br>Rx interrupts: 11089<br>Tx interrupts: <some number in hundreds> | Demonstrates that large numbers of inputs don't overload the system. Also, the TX interrupts are reliably much less than RX interrupts. |
| Send characters rapidly (holding a key down) in byte mode | All characters are echoed. RX and TX counts remain equal with no missed interrupts. | Validates that the interrupt-driven RX path and TX queue can sustain high-frequency input without data loss or counter mismatch. |

**<u>Answer To Questions:</u>**

**1.** Without critical section protection like *taskENTER_CRITICAL(),* a hardware interrupt could fire while a driver function is in the middle of modifying a shared resource, such as the software queues or UART hardware registers. This can lead to race conditions, corrupted data strings, or system crashes if the ISR and a user task try to manipulate the same memory simultaneously.

**2.** Yes, there is a preferred order for processing interrupts. Receive (RX) interrupts should generally be prioritized over transmit (TX) interrupts. If an RX interrupt is delayed, the hardware's receive FIFO can overflow, resulting in permanent data loss. Conversely, if a TX interrupt is delayed, the outgoing data simply sits in the software queue for a few microseconds longer. Delaying transmission is non-destructive, whereas delaying reception compromises data integrity or even the system integrity.

**3.** The *TXEMPTY* interrupt triggers whenever the hardware transmit FIFO is empty. If this interrupt is left enabled when *xTxQueue* has no more data to supply, the interrupt will fire continuously. This creates an infinite loop where the CPU spends all its time servicing an interrupt for an empty FIFO, starving the FreeRTOS scheduler, halting user tasks, and effectively freezing the entire system. In other words the system wasted doing unnecessary/useless work.

**4.** When typing a short message and using *mySendByte(),* each character is sent individually. This causes the FIFO to empty every time, triggering a new TX interrupt for every single byte. Thus, the TX and RX counts are equal. On the other hand, when typing a medium or long message, the system queues up all the characters in xTxQueue. The first byte triggers the ISR, which then pulls multiple bytes off the queue and loads them into the hardware FIFO in a single interrupt cycle. This batches the transmissions, resulting in significantly fewer TX interrupts compared to RX interrupts with it being substantially more evident the longer the message. When it comes to using *mySendString* regardless of the size we find that (like a longer message using *mySendByte)* the RX and TX interrupt counts diverge. This is for the same reason as prior.

**5.** Making queues and hardware details private ensures that all UART communication passes exclusively through the driver abstraction layer. This prevents parts of the application from accidentally bypassing the queues and accessing the hardware directly, which could corrupt the driver state. Encapsulating these details maintains a clean separation between the application layer and the hardware layer, ensuring consistent behavior, proper resource management, and a much more robust system. In other words, it is crucial for maintaining security, integrity and simplicity for the user and system.

**Conclusion:**

The objective of this second lab was to gain familiarity with UART interfacing, hardware interrupts, and utilizing FreeRTOS queues for decoupling system tasks. This took place in three parts, starting with a polling-based SHA-256 text hasher, moving to a command-based peripheral controller using our Lab 1 components, and culminating in a highly efficient interrupt-driven UART driver. All in all, this lab was a big success. We were able to achieve functionality across all aspects, gaining a newfound understanding of asynchronous hardware communication, avoiding CPU-hogging polling loops, and properly handling hardware FIFOs via ISRs. Although we came across some difficulties in the lab, particularly with part three, we successfully overcame them. Ultimately, this ECE 315 lab was done in full and was a complete success.

**References:**

1. Provided lab documentation and source code
2. Inclass notes
3. FreeRTOS Documentation; https://freertos.org/Documentation/00-Overview
4. Test matrix generated using ClaudeAI
5. Made with help from Gemini Pro